

以下是为您总结的各份文档中主要讲解的问题清单：

- **CPT102_2526_Exercise 9 soln.pdf (深度优先搜索与广度优先搜索)** 主要讲解如何使用 DFS 和 BFS 来解决图论问题：
 - **环检测 (Cycle Detection)**: 在无向图中判断是否存在环，并输出环的顶点。
 - **二分图/双色性 (Bipartite Graph / Two-colorability)**: 判定图是否为二分图，如果不是则找出奇数长度的环。
 - **连通分量 (Connected Components)**: 利用 DFS 计算图中所有的连通分量并获取其大小。
 - **双连通性与割点 (Biconnectivity)**: 利用 DFS 的发现时间 (pre) 和最早可达时间 (low) 寻找割点 (Articulation vertex)，判断图是否双连通。
 - **树的直径 (Diameter of a Tree)**: 通过两次 BFS 调用来寻找树中最长路径的两个端点。
- **CPT102_2526_Exercise 10 soln.pdf (最短路径与最小生成树)**
 - **允许跳过一条边的最短路径**: 修改 Dijkstra 算法 (正反向各运行一次)，寻找允许将某一条边权重改为 0 时的最短路径。
 - **边不相交的路径 (Edge Disjoint Paths)**: 寻找两条边不重叠的最小权重路径 (通过构建残余图并运行两次 Dijkstra 算法)。
 - **最大生成树 (Maximum Spanning Trees)**: 通过修改 Prim 算法求解最大生成树。
 - **最小生成森林 (Minimum Spanning Forest)**: 修改 Kruskal 或 Prim 算法处理不连通图的生成森林问题。
- **CPT102_2526_Exercise 11 soln.pdf (图论问题求解)**
 - **计算房间数 (Counting Rooms)**: 将网格转化为图，使用 DFS/BFS 计算连通分量的数量。
 - **DAG 中的哈密顿路径**: 利用拓扑排序 (Topological sort) 寻找有向无环图中是否包含访问每个顶点恰好一次的路径。
 - **奇数长度的有向环**: 结合强连通分量 (SCC) 算法和二分图检测算法来判定图中是否有奇数长度的有向环。
 - **Kosaraju 算法的正确性**: 证明利用两次 DFS 求解强连通分量的 Kosaraju 算法原理。

- **CPT102_2526_Exercise 12 soln.pdf (二叉搜索树与平衡搜索树)**
 - **查找最大值与最小值**: 在二叉搜索树 (BST) 中寻找最左侧或最右侧节点的算法实现。
 - **删除最小/最大节点**: 在普通 BST 中删除极值节点并更新树结构及大小。
 - **红黑树中的删除操作**: 在红黑树中实现删除最小节点 (Delete Minimum), 要求在沿左侧向下移动时保持不出现 2- 节点的性质。
- **CPT102_2526_Exercise 13 Solution.pdf (哈希 Hashing)**
 - **哈希冲突 (Hashing Collision)**: 基于特定的哈希算法构造拥有相同哈希值的长字符串 (例如利用模式拼接)。
 - **哈希与搜索**: 利用预计算的哈希值来加速链表中长字符串的比较与搜索。
 - **分离链接哈希表的删除操作**: 编写分离链接法 (Separate chaining) 哈希表删除指定键的伪代码。
- **week2_exercises_solutions.pdf (基于数组的基础算法及复杂度分析)**
 - **基础数组算法**: 计算数组平均值、检查数组是否存在重复项。
 - **统计数组中不同值的数量 (Counting distinct values)**: 探讨了一般情况、排序后情况以及有界整数 (利用布尔数组) 等不同前提下的时间复杂度优化算法。
 - **最大子数组和 (Maximum sub-array sum)**: 从暴力破解、前缀和优化, 逐步递进到时间复杂度为线性的最佳解法——Kadane 算法。
 - **括号平衡表达式检测 (Well-balanced expressions)**: 使用计数器 (针对单一括号) 或栈 (针对多类括号) 检测表达式的匹配合法性。
- **week3_exercises_solutions.pdf (列表与队列的底层实现)**
 - **List 操作实现**: 在 ArrayList、LinkedList 以及双向链表 (Doubly-linked list) 结构上实现 Contains (查找) 和 Remove (基于索引删除) 的方法及时间复杂度分析。
 - **Queue 数据结构**: 分别实现了基于链表的队列 (LinkedQueue) 和基于数组的队列 (ArrayQueue) 的各种标准操作 (如 Enqueue, Dequeue, Peek)。
 - **循环队列 (CircularArrayQueue)**: 探讨了数组队列中头尾指针平移造成

的空间浪费，并给出了包含扩容机制的循环数组队列实现。

- **week5_tutorial.pdf (Comparable 接口与选择排序)**

- **Comparable 接口**：Java 中基于 `compareTo()` 方法实现的对象排序契约（如建立大小关系以支持列表排序）。
- **选择排序 (Selection sort)**：选择排序的核心原理、代码实现，并分析了其时间复杂度 ($O(n^2)$) 和不稳定性 (instability)。
- **实验任务**：要求通过编写代码对整数成绩数组进行选择排序，并让自定义的 `Student` 类实现 `Comparable` 接口进行对象级别的排序。

- **week6_tutorial.pdf (归并排序 Merge Sort)**

- **分治思想**：归并排序的分解 (Divide)、求解 (Conquer) 与合并 (Combine) 原理。
- **合并操作 (Merge operation)**：详细剖析了如何将两个已经有序的子数组合并为一个有序数组的完整过程。
- **实验任务**：要求修改基础算法使之支持泛型对象或浮点数的归并排序，并使用静态变量统计 Merge 操作的调用次数以验证其复杂度随输入规模翻倍时的增长规律。